

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÀI TẬP LỚN OOP

Đề tài: Bros || Crush

Sinh viên thực hiện:	MSSV
Nguyễn Quang Đức (nhóm trưởng)	20235044
Lê Đức Chính	20235023
Trần Gia Định	20235036
Nguyễn Bá Đức Anh	20235004
Hồ Minh Dũng	20235050

Giáo viên hướng dẫn: Trần Nhật Hoá
Nhóm: 15
Mã lớp: 153989

Mục lục

I. Đề tài	4
1. Mô tả chung về đề tài	4
2. Thể loại	4
II. Phân công công việc và đánh giá	4
III. Nội dung bài tập lớn	5
• Các tương tác giữa các package của dự án	5
• Các tương tác giữa các lớp trong các package	7
- Lớp GameWorld	7
1. Package manager	7
2. Package scences	7
3. Package screens	8
4. Package sprites	8
4.1. Package effectedObject	8
4.2. Package enemy	9
4.3. Package items	9
4.4. Package trap	10
4.5. Package weapons	10
5. Package tools	10
6. Package UI	11
6.1. Package description	11
6.2. Package lastlevel	11
6.3. Package story	11
IV. Áp dụng các kỹ thuật lập trình hướng đối tượng	11
1. Tính kế thừa	11
2. Tính đóng gói	12
3. Tính trừu tượng	12
4. Tính đa hình	12
V. Những gì học được sau project	13

I. Đề tài

1. Mô tả chung về đề tài

- Người chơi vào vai nhân vật Chính, một sinh viên đại học phải đối mặt với nhiều thử thách để giải cứu công chúa Linh khỏi bàn tay của phù thủy Eyesightasu.
- Người chơi điều khiển Chính qua nhiều màn chơi khác nhau, đối đầu với nhiều loại quái vật đa dạng. Sau khi chiến thắng boss ở màn cuối cùng, người chơi sẽ phải đưa ra lựa chọn giữa việc giải cứu công chúa Linh hoặc ba người bạn thân đã hy sinh trong hành trình giải cứu công chúa.
- Kết quả mong đợi của dự án là một trò chơi điện tử 2D có đồ họa đơn giản nhưng cuốn hút, cốt truyện sáng tạo, hài hước, và mang đến trải nghiệm nhập vai thú vị.

2. Thể loại

- Game 2D

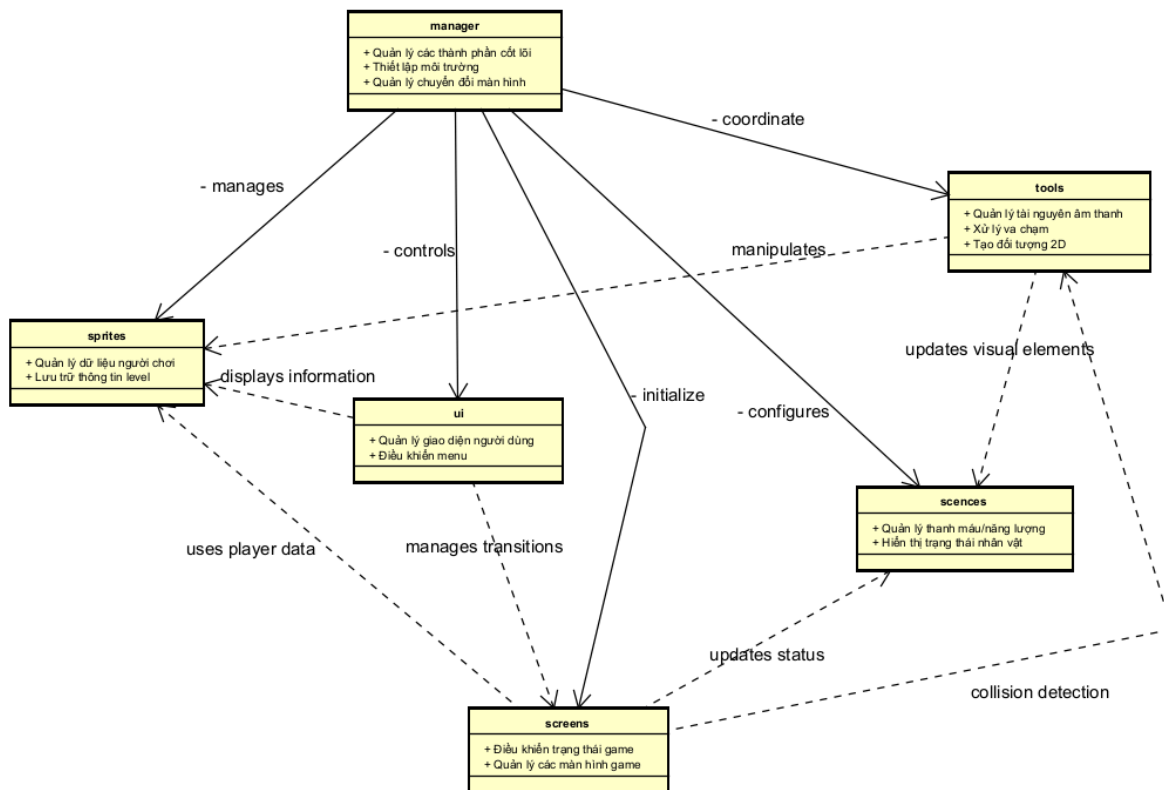
II. Phân công công việc và đánh giá

MSSV	Họ và tên	Nhiệm vụ	Đánh giá
20235044	Nguyễn Quang Đức	Phân công và hỗ trợ công việc, xây dựng và hoàn thiện base code, phát triển UI, hoàn thiện logic tổng thể và tích hợp toàn bộ các màn chơi để đảm bảo game hoạt động mượt mà và không có lỗi.	Hoàn thành tốt, đảm bảo tiến độ và chất lượng, hỗ trợ nhóm tốt. (10đ)
20235023	Lê Đức Chính	Hoàn thành toàn bộ code cho màn 2, phát triển các logic đặc biệt như thanh máu cho nhân vật và enemy, làm logic công chúa, làm video giới thiệu project.	Hoàn thành tốt, có khả năng giải quyết vấn đề và hỗ trợ đồng đội. (10đ)
20235036	Trần Gia Định	Hoàn thành toàn bộ code cho màn 1, xử lý các logic cơ bản, đồng thời chịu trách nhiệm viết và hoàn thiện báo cáo dự án một cách đầy đủ và chuyên nghiệp.	Làm việc hiệu quả, hỗ trợ đồng đội và thực hiện tốt nhiệm vụ được giao. (10đ)
20235004	Nguyễn Bá Đức Anh	Phát triển và hoàn thiện toàn bộ code cho màn 4 (màn boss), xử lý các logic đặc biệt như trận chiến với boss. Ngoài ra, thực hiện slide thuyết trình cho dự án.	Hoàn thành tốt, đặc biệt trong việc xử lý logic phức tạp. (10đ)

20235050	Hồ Minh Dũng	Hoàn thành toàn bộ code cho màn 3, xây dựng các logic tương tác vật lý, logic di chuyển của nhân vật (player) và logic load map. Đồng thời, tìm kiếm và tích hợp các tài nguyên như animation, hòm, và các assets trong game.	Hoàn thành tốt, hỗ trợ đồng đội và thực hiện tốt nhiệm vụ được giao. (10đ)
----------	--------------	---	---

III. Nội dung bài tập lớn

- Các tương tác giữa các package của dự án



- Mũi tên liên đặc (-->) biểu thị mối quan hệ "association" (liên kết) biểu thị một mối quan hệ cố định giữa các lớp. Mũi tên (nếu có) chỉ chiều điều hướng của quan hệ.
Ví dụ: Manager --> Screens nghĩa là Manager khởi tạo và điều khiển Screens
- Mũi tên chấm (..>) biểu thị mối quan hệ "dependency" (phụ thuộc): một thành phần phụ thuộc vào một thành phần khác (một package sử dụng chức năng hoặc dữ liệu của package khác)
Ví dụ: Screens ..> Sprites nghĩa là Screens phụ thuộc vào dữ liệu từ Sprites

- Các tương tác giữa các lớp trong các package
- Lớp GameWorld

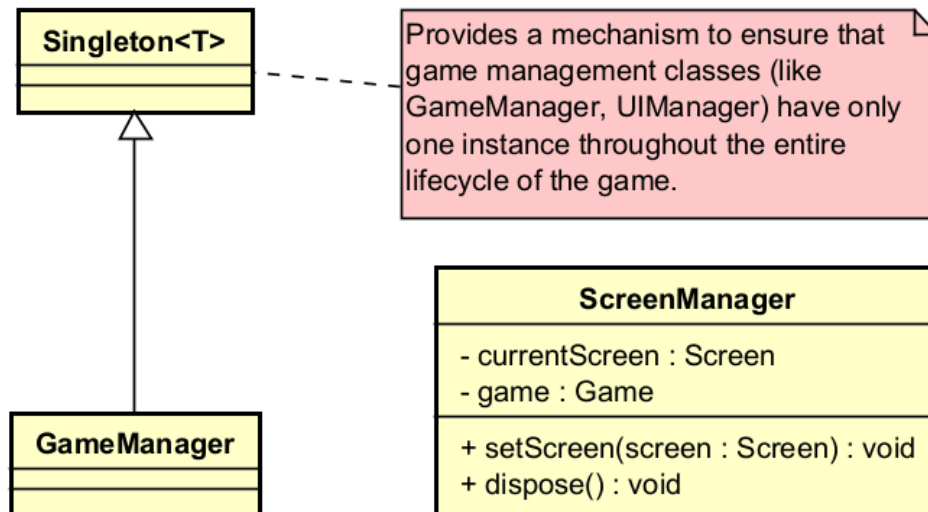
Lớp kế thừa từ lớp Game của LibGDX, đóng vai trò quản lý thế giới game và các thuộc tính chung của trò chơi. Đây là nơi cấu hình các thuộc tính cơ bản của thế giới trò chơi và bắt đầu các màn hình (screens) của game.

- Lớp Main

Kế thừa từ Game và chịu trách nhiệm khởi tạo và quản lý game ở cấp độ cao hơn. Lớp này quản lý việc khởi tạo các manager và UI.Package manager

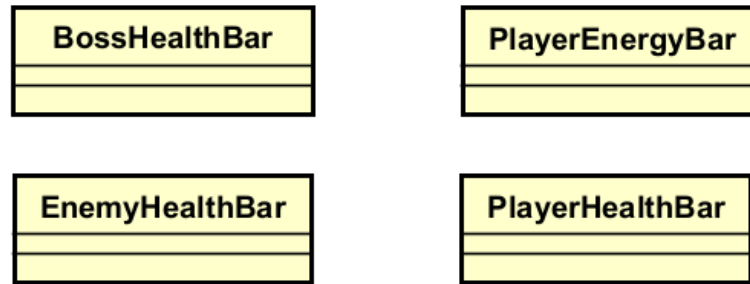
1. Package manager

Quản lý các thành phần cốt lõi và logic tổng thể, bao gồm: thiết lập môi trường (âm thanh, tạo đối tượng 2D, vẽ map,...), quản lý chuyển đổi các màn hình trò chơi,...



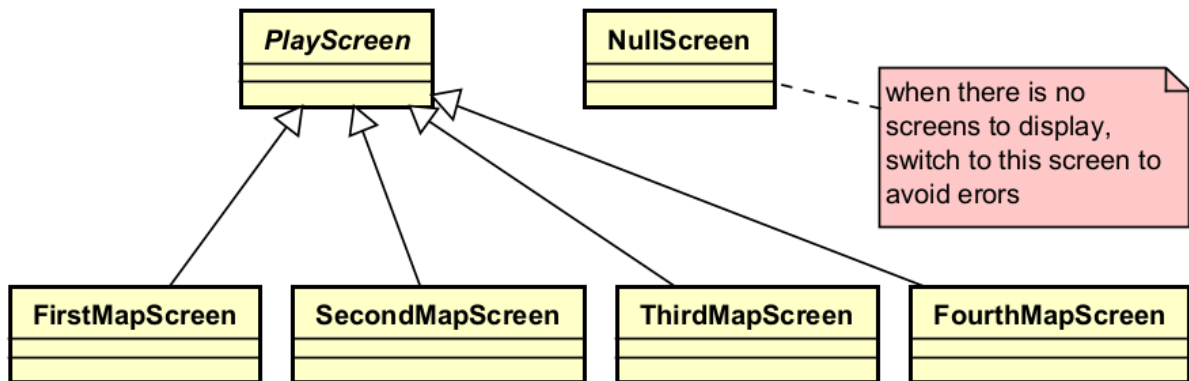
2. Package scences

Quản lý và hiển thị thanh máu (hoặc năng lượng) của nhân vật chính, boss và quái thường trong game.



3. Package screens

Quản lý các màn hình khác nhau trong trò chơi. Chịu trách nhiệm tổ chức các lớp đại diện cho từng trạng thái hoặc giai đoạn cụ thể của game, từ giao diện chính, màn chơi đến màn hình kết thúc game.



4. Package sprites

- Lớp Knight

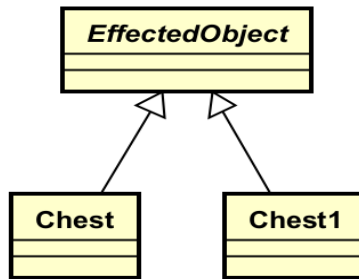
Lớp này quản lý dữ liệu của người chơi, bao gồm các thông tin về mức độ hoàn thành các level, điểm số, và trạng thái của các level trong game.

- Lớp PlayerData

Lớp này quản lý dữ liệu của người chơi, bao gồm các thông tin về mức độ hoàn thành các level, điểm số, và trạng thái của các level trong game.

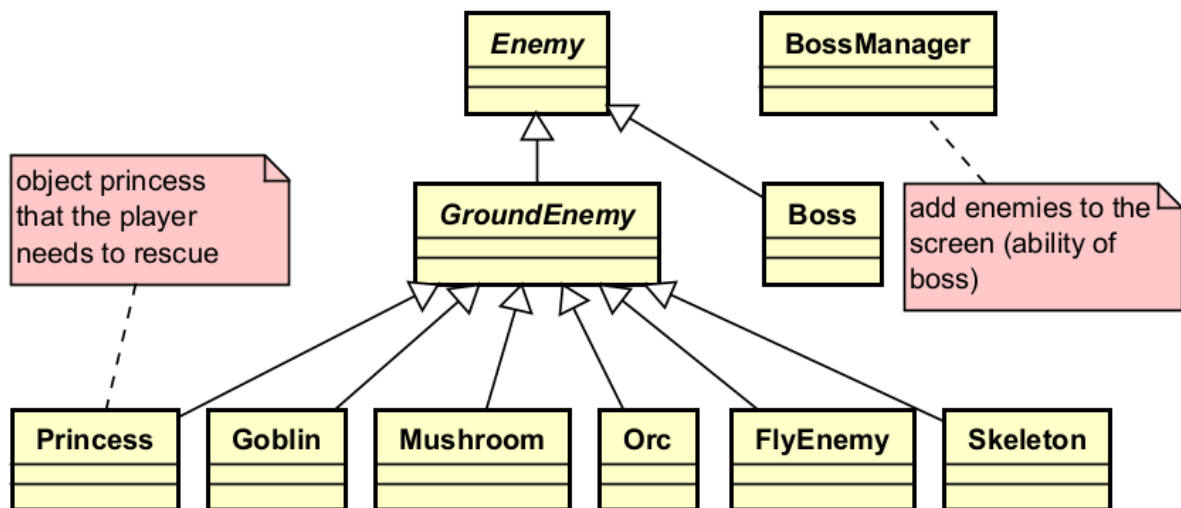
4.1. Package effectedObject

Đại diện cho các đối tượng trong game có thể bị tác động và thay đổi trạng thái.



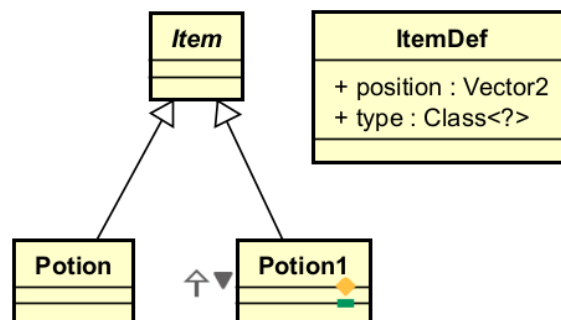
4.2. Package enemy

Định nghĩa các cấu trúc và hành vi của các loại kẻ địch trong trò chơi.



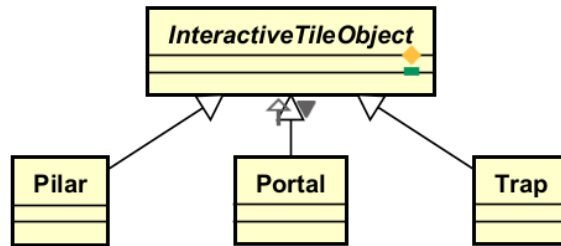
4.3. Package items

Đại diện cho các vật phẩm mà người chơi có thể tương tác hoặc thu thập.



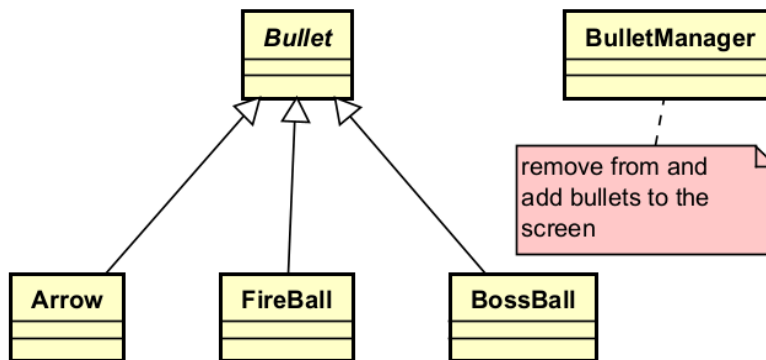
4.4. Package trap

Định nghĩa các đối tượng tương tác trong bản đồ ô vuông (tiled map) của trò chơi. Đây là các đối tượng có thể tác động đến người chơi/quái vật khi va chạm.



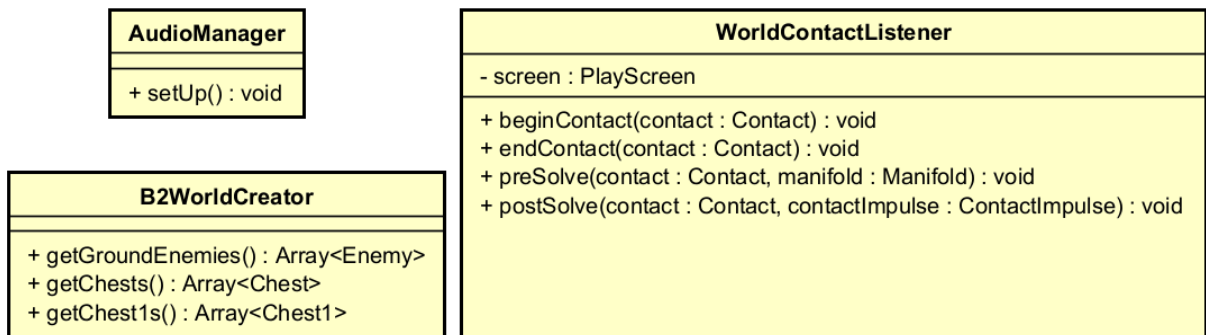
4.5. Package weapons

Định nghĩa các loại đạn trong trò chơi. Định nghĩa các thuộc tính và hành vi cơ bản của một viên đạn cụ thể (VD: Arrow, BossBall,...)



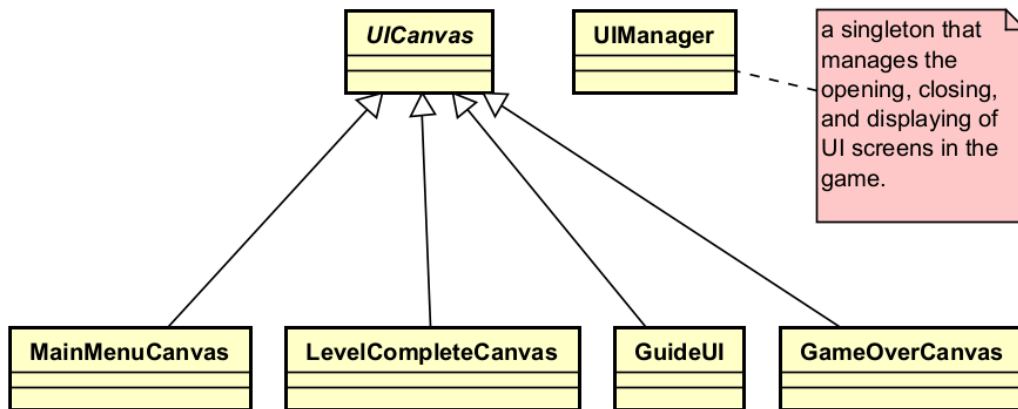
5. Package tools

Quản lý tất cả các tài nguyên âm thanh, tạo các đối tượng 2D và xử lý va chạm, tương tác trong trò chơi.



6. Package UI

- Lớp UIManager

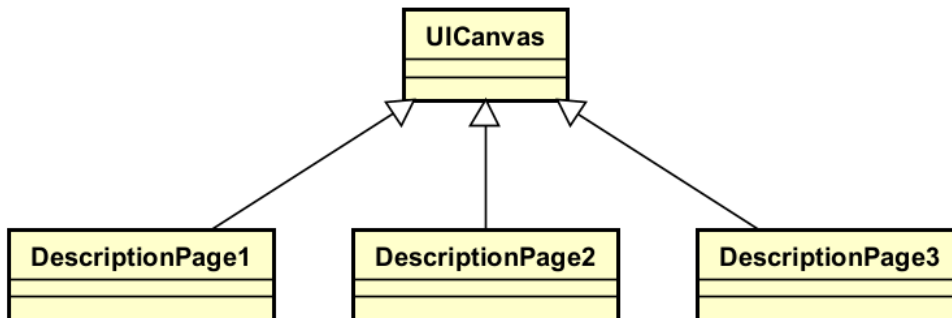


- Lớp UI canvas

Là lớp cơ sở cho các màn hình UI trong game. Các màn hình UI sẽ kế thừa lớp này và tùy chỉnh để hiển thị giao diện người dùng. Việc mở/đóng các popup cũng được quản lý linh hoạt.

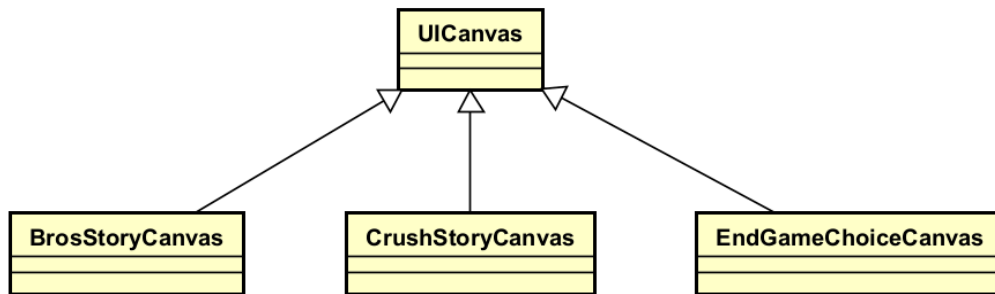
6.1. Package description

Chịu trách nhiệm tạo trang giao diện mô tả trong game, nơi người chơi có thể xem thông tin hoặc chuyển đổi giữa các trang mô tả khác.



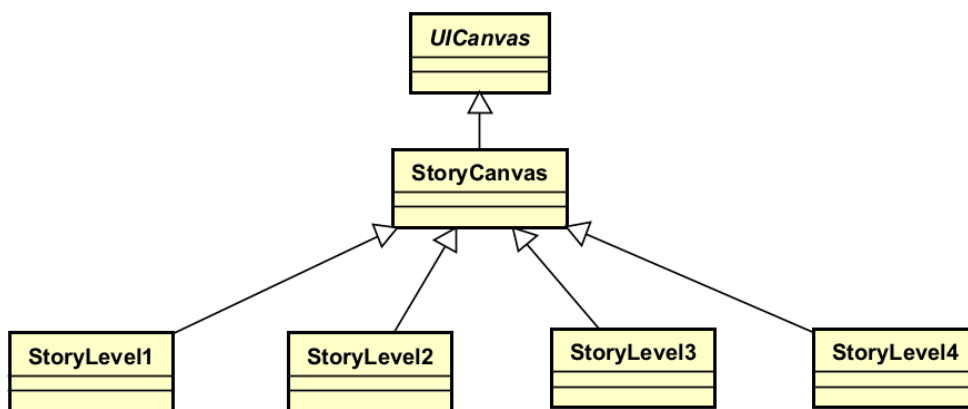
6.2. Package lastlevel

Phần UI cho phép người chơi lựa chọn kết thúc mở: giải cứu đồng đội hoặc công chúa.



6.3. Package story

Bao gồm các giao diện cho các giai đoạn khi người chơi thua cuộc, hướng dẫn chơi, chuyển đổi giữa các màn, thanh công cụ menu,...



IV. Áp dụng các kỹ thuật lập trình hướng đối tượng

1. Tính kế thừa

1) Lớp Boss kế thừa từ lớp Enemy

```

public class Boss extends Enemy { 4 usages  📄 Nguyen Quang Duc

    protected float scaleX; 2 usages
    protected float scaleY; 2 usages
  
```

2) Lớp Pilar kế thừa lớp InteractiveTileObject

```

public class Pilar extends InteractiveTileObject { 5 usages  📄 Nguyen Quang Duc
    public Pilar(PlayScreen screen, MapObject object) { 6 usages  📄 Nguyen Quang Duc
        super(screen, object);
    }
  
```

2. Tính đóng gói

Đối tượng player trong lớp PlayScreen là protected và ta cần phương thức getter getPlayer() để truy cập vào đối tượng này trong một số trường hợp

```
public abstract class PlayScreen implements Screen { 4 inheritors  ⬆️ Nguyen Quan
    // test
    protected Knight player;

    public Knight getPlayer() { return player; }
```

3. Tính trừu tượng

Lớp Enemy có các phương thức trừu tượng dưới đây để định hình các phương thức cho các lớp con của mình

```
protected abstract void defineEnemy(); 1 usage 3 implementations  ⬆️ Nguyen Quang Duc
protected abstract void prepareAnimation(); 6 implementations  ⬆️ Nguyen Quang Duc
protected abstract void prepareAudio(); 1 usage 2 implementations  ⬆️ Nguyen Quang Duc
public abstract void update(float dt); 3 implementations  ⬆️ Nguyen Quang Duc
public abstract void hitOnHead(); no usages 2 implementations  ⬆️ Nguyen Quang Duc
public abstract void destroy(); 2 implementations  ⬆️ Nguyen Quang Duc
public abstract void attackingCallback(); 2 usages 2 implementations  ⬆️ Nguyen Quang Duc
public abstract void hurtingCallback(); 6 usages 2 implementations  ⬆️ Nguyen Quang Duc
```

4. Tính đa hình

Các goblins là các thể hiện của lớp Goblins được coi như là các Enemy (lớp Goblins kế thừa từ lớp Enemy) khi thêm vào Array groundEnemies

```
public class B2WorldCreator { 10 usages  ⬆️ Nguyen Quang Duc
    private Array<Enemy> groundEnemies; 17 usages
```

```
//create all goblins
for (MapObject object : map.getLayers().get(13).getObjects().getByType(RectangleMapObject.class)) {
    Rectangle rect = ((RectangleMapObject) object).getRectangle();
    groundEnemies.add(new Goblin(screen, x: rect.getX() / GameWorld.PPM, y: rect.y / GameWorld.PPM));
}
```

V. Những gì học được sau project

Qua quá trình thực hiện dự án game "Bro || Crush", nhóm em đã học được rất nhiều điều về lập trình hướng đối tượng (OOP) và cách áp dụng các kỹ thuật trong thực tế. Cụ thể, một số bài học quan trọng bao gồm:

1. Kỹ thuật lập trình hướng đối tượng (OOP):

- **Tính kế thừa:** Chúng em đã sử dụng tính kế thừa để tổ chức mã nguồn, giúp tái sử dụng các lớp cơ bản như lớp Enemy và lớp Boss, giảm thiểu sự lặp lại mã và làm cho hệ thống dễ dàng mở rộng hơn.
- **Tính đóng gói:** Các đối tượng trong game, chẳng hạn như nhân vật người chơi, đã được đóng gói để bảo vệ dữ liệu và chỉ cung cấp những phương thức cần thiết để tương tác.
- **Tính trừu tượng:** Việc sử dụng các lớp trừu tượng cho các đối tượng như Enemy giúp định hình và chuẩn hóa hành vi của các đối tượng này, tạo ra sự linh hoạt khi thêm các loại quái vật mới.
- **Tính đa hình:** Các đối tượng thuộc lớp Goblin, dù khác biệt về hành vi, đều được xử lý như các Enemy, giúp hệ thống trở nên đơn giản và dễ mở rộng khi thêm các loại kẻ địch khác.

2. Áp dụng Design Patterns:

- Chúng em đã sử dụng một số **design patterns** để tối ưu hóa và quản lý mã nguồn hiệu quả. Một số patterns quan trọng được áp dụng là:
 - **Singleton Pattern:** Được sử dụng cho các lớp quản lý game như GameManager và UIManager, đảm bảo chỉ có một thể hiện duy nhất trong suốt quá trình chạy game, giúp quản lý trạng thái toàn cục của game một cách dễ dàng.
 - **Observer Pattern:** Được sử dụng trong **WorldContactListener** để xử lý các tương tác vật lý giữa các đối tượng. Khi có sự kiện va chạm xảy ra, WorldContactListener sẽ thông báo đến các đối tượng liên quan (như Player, Enemy), giúp chúng cập nhật trạng thái như trừ máu hoặc kích hoạt hiệu ứng.

3. Làm việc nhóm và quản lý dự án:

Qua việc phân công công việc và phối hợp chặt chẽ giữa các thành viên, chúng em đã học được cách làm việc hiệu quả trong nhóm, giúp các thành viên phát huy được thế mạnh của mình. Việc phối hợp và hỗ trợ nhau trong quá trình phát triển game giúp chúng em hoàn thành dự án đúng tiến độ và với chất lượng cao.

4. Phát triển kỹ năng lập trình game:

Đây là lần đầu tiên chúng em phát triển một trò chơi hoàn chỉnh, chúng em đã học được nhiều kỹ thuật từ việc tạo đồ họa 2D đơn giản, xử lý các tương tác vật lý, đến việc tối ưu mã nguồn và bảo trì game trong suốt quá trình phát triển.

